

- سوال ۱: چرا هنگام اجرای یک اسکریپت در دایرکتوری فعلی، باید از `script_name.sh` استفاده کنیم و نمی‌توانیم فقط `script_name.sh` را بنویسیم؟ (راهنمایی: به متغیر سیستمی `PATH` فکر کنید).

script_name.sh

شل (مثل bash) دنبال این فایل در **مسیرهای موجود در متغیر محیطی PATH** می‌گردد، نه در پوشه‌ی فعلی. و معمولاً دایرکتوری فعلی (.) به دلایل امنیتی داخل **PATH** نیست.

چرا؟

مثلاً داخل یک پوشه باشیم که یک **فایل مخرب** به نام **ls** یا **python** در آن وجود دارد. اگر (.) در **PATH** باشد

و بنویسیم **ls**، ممکن است به جای **/bin/ls**، همان **فایل مخرب** اجرا شود.

- سوال ۱: چرا هنگام اجرای یک اسکریپت در دایرکتوری فعلی، باید از `./script_name.sh` استفاده کنیم و نمی‌توانیم فقط `script_name.sh` را بنویسیم؟ (راهنمایی: به متغیر سیستمی `$PATH` فکر کنید).

`./script_name.sh`

این فایل را از همین دایرکتوری فعلی اجرا کن

مسیر فایل را صریح مشخص می‌کند ، پس دیگر نیازی به `PATH` نیست

- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.

```
#!/usr/bin/env bash

read -p "Enter the length: " length
read -p "Enter the width: " width

re='^[0-9]+([.][0-9]+)?$'

if ! [[ $length =~ $re && $width =~ $re ]]; then
    echo "Error: length and width must be numeric values ."
    exit 1
fi

area=$(awk "BEGIN {print $length * $width}")
perimeter=$(awk "BEGIN {print 2 * ($length + $width)}")

echo "Area: $area"
echo "Perimeter: $perimeter"
```

- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.

`#!/usr/bin/env bash`

به سیستم عامل می‌گوید این فایل را با چه برنامه‌ای اجرا کند

`/usr/bin/env bash` یعنی «برنامه‌ی bash را از داخل PATH پیدا کن و با آن این اسکریپت را اجرا کن»

چرا این روش خوب است؟

چون در بعضی سیستم‌ها ممکن است `bash` دقیقاً در `/bin/bash` نباشد، ولی `env` آن را پیدا می‌کند پس این روش قابل حمل تر است.

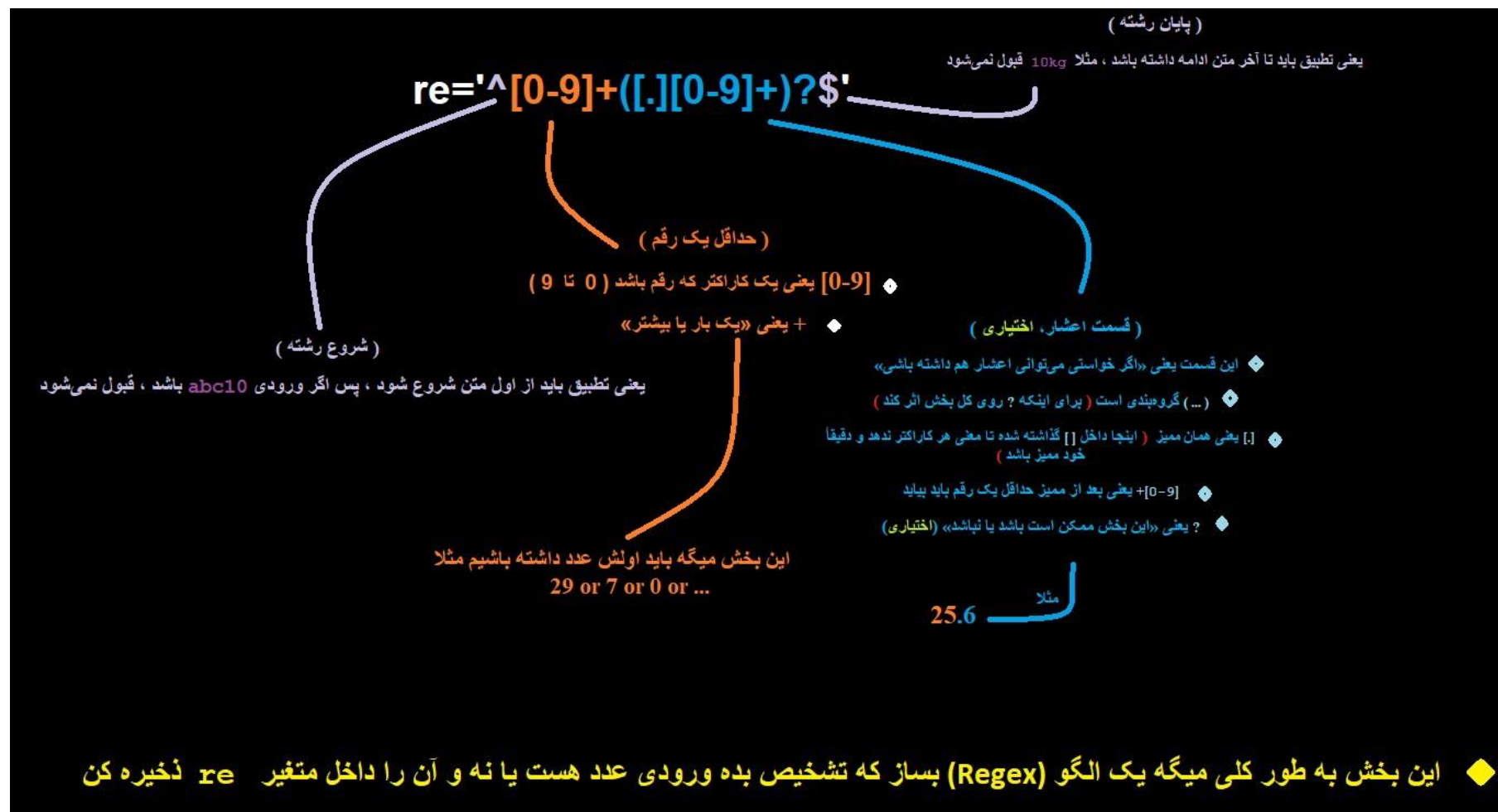
- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.

```
read -p "Enter the lenght: " length  
read -p "Enter the width: " width
```

گرفتن طول از کاربر

گرفتن عرض از کاربر

- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.



- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.

```
if ! [[ $length =~ $re && $width =~ $re ]]; then
    echo "Error: length and width must be numeric values."
    exit 1
fi
```

! اگر هر کدام درست نبود ؛ یعنی ورودی نامعتبر بود برو داخل then

هر دو باید درست باشند

`[[ $width =~ $re ]]` یعنی width با regex ، معج میشود؟

1 یعنی خروج با خطا
عرف سیستم ها 0 یعنی موفقیت ، غیر صفر یعنی خطا

یعنی اسکریپت را تمام کن

پایان بلوک

اگر هر کدام از ورودی ها عدد نباشد، این پیام نمایش داده میشود

`[[ $length =~ $re ]]` یعنی length با regex ، معج میشود؟

این if اجرای بررسی و کنترل جریان برنامه است

- سوال ۲: اسکریپتی بنویسید که طول و عرض یک مستطیل را از کاربر بگیرد و محیط و مساحت آن را محاسبه و چاپ کند.

مقدار **length** را در **width** ضرب کن و نتیجه را داخل متغیر **area** ذخیره کن

$Area = Length \times Width$

محیط

```
area=$(awk "BEGIN {print $length * $width}")
```

مساحت

```
perimeter=$(awk "BEGIN {print 2 * ($length + $width)}")
```

length و **width** را جمع کن ، حاصل را در 2 ضرب کن و نتیجه را داخل متغیر **perimeter** ذخیره کن

$Perimete = 2 \times (Length + Width)$

awk میتواند اعداد را حساب کند

(**bash** به صورت پیشفرض در محاسبات ، با اعداد خوب کار نمیکند)

در **awk** ، بخش **BEGIN** یعنی:

این دستور را همان ابتدا اجرا کن، بدون اینکه لازم باشد فایلی خوانده شود

معمولا **awk** برای پردازش خط به خط فایل ها استفاده می شود ، اما وقتی فقط محاسبه می خواهیم ، از **BEGIN** استفاده می کنیم

- سوال ۳: متغیرهای "\$*" و "\$@" در اسکریپت‌نویسی چه تفاوتی با هم دارند؟ (در مورد "آرگومان‌های خط فرمان" یا "Command-line arguments" تحقیق کنید). این مبحث، مقدمه‌ای برای جلسه بعدی خواهد بود.

Using :	What does it do ?
"\$*"	Concatenate all arguments and create a single string
"\$@"	Keep arguments separate as given by the user